Kamloops-daily-meantemp

2026 March 04

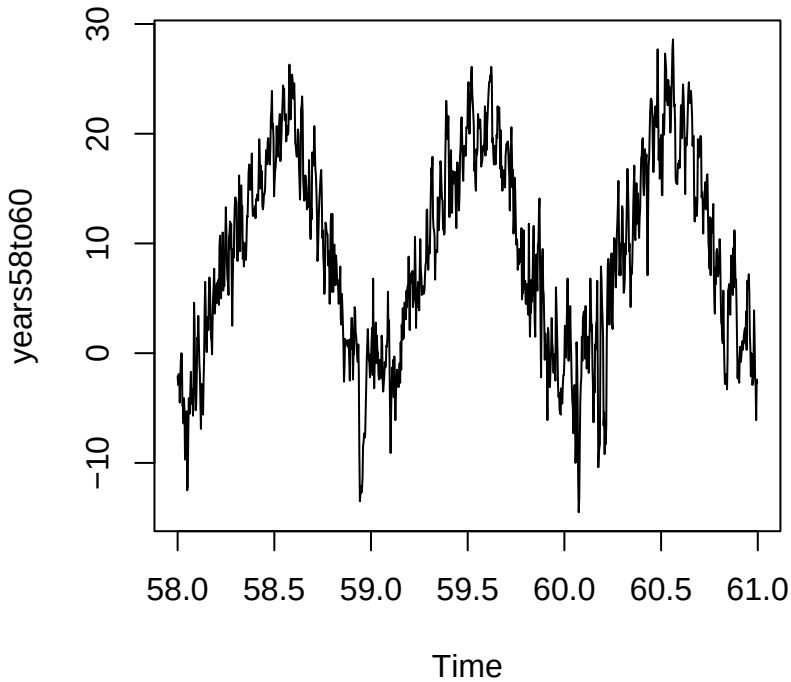
Step 1: Original source plus Wrangling

```
# Canadian daily weather data; source is: https://climate.weather.gc.ca
# Available years of data depends on location;
# 10000 days can be downloaded at one time.
# The station within a city location may have changed,
# and with the change the variables may be in permuted columns.

# It takes wrangling to get the following from 5 separate downloads.

# Daily data for 84 years (>30000 days) from 1940 and 2023 but some years have missing segments
# Isolated missing values for maxtemp, mintemp, meantemp were imputed with average of neighbours,
# around 91 cases over 84 years

# The csv file in the next line is at the course web site.
kamloops1 = read.csv("Kamloops-temperature-1940to2023.csv", header=T)

names(kamloops1)
#> [1] "Date"     "totprec"  "maxtemp"  "mintemp"  "meantemp" "year"     "month"
#> [8] "day"

print(dim(kamloops1))
#> [1] 30523     8

# Delete the leap day Feb 29
ifeb29 = (kamloops1$month==2 & kamloops1$day==29)
print(sum(ifeb29))
#> [1] 21

kamloops2 = kamloops1[!ifeb29,]
print(dim(kamloops2))
#> [1] 30502     8

table(kamloops2$year)
#>
#> 1940 1941 1942 1943 1944 1945 1946 1947 1948 1949 1950 1951 1952 1953 1954 1955
#>  365  365  365  365  365  365  365  365  365  365  365  365  365  365  365  365
#> 1956 1957 1958 1959 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 1970 1971
#>  365  365  365  365  365  365  365  365  365  365  365  365  365  365  365  365
#> 1972 1973 1974 1975 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987
#>  365  365  365  365  365  365  365  365  365  365  365  365  365  365  365  365
#> 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003
#>  365  364  365  365  346  235  365  365  365  365  365  365  365  365  365  365
#> 2004 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019
```

```r
#>  365  365  365  365  365  365  365  365  365  357  365  365  365  365  365  365
#> 2020 2021 2022 2023
#>  365  365  365  365

# 1989 has 364, 1992 has 346, 1993 has 235, 2013 has 357,
# Delete these 4 years
iyrdel = (kamloops2$year==1989 | kamloops2$year==1992 | kamloops2$year==1993 | kamloops2$year==2013)
print(sum(iyrdel))
#> [1] 1302

kamloops_df = kamloops2[!iyrdel,]

print(dim(kamloops_df))
#> [1] 29200     8

print(nrow(kamloops_df)/365)
#> [1] 80


# Save wrangled data into RData file so that all further analysis makes use of this
# This step is very important for a team project; from this data set,
#  team members can apply different time series and forecasting methods

save(file="01kamloops.RData", kamloops_df)
```

Step 2: Set up ts objects for training and holdout sets

```r
# load("01kamloops.RData")

# For ts object for meantemp, replace by years 1 to 80 because some years are omitted

kamloops_ts = ts(kamloops_df$meantemp, start=c(1,1), frequency=365)

# Use first 60 years as training and last 20 as holdout (ts objects)
train = window(kamloops_ts,end=c(60,365))
print(length(train)/365)
#> [1] 60

holdout = window(kamloops_ts,start=c(61,1))

years58to60 = window(kamloops_ts,start=c(58,1),end=c(60,365))
plot(years58to60)
```




```
# Keep only the meantemp variable for forecasting.
save(file="02kamloops.RData", kamloops_ts, train, holdout)
```

Step 3: Compare simple forecast methods,

Some rmse functions for forecasting included from course webs site

```
# load("02kamloops.RData")

#' @description forecast rule: persistence by period
#' @param train training set with length ntrain
#' @param holding holdout set with length nholdout
#' @param periodicity 4 for quarterly, 12 for monthly,  365 for daily
#' @return list with holdout set rmse, holdout set forecast vector
persistbyperiod_fc = function(train, holdout, periodicity)
{ ntrain = length(train)
  nholdout = length(holdout)
  m = periodicity
  sse = 0
  fc_vec = numeric(nholdout)
  full_data = c(train, holdout)
  for (i in 1:nholdout)
  { yt = holdout[i]
    fc = full_data[(ntrain+i)-m]
```

```
    fc_vec[i] = fc
    fcerror = yt - fc
    sse = sse + fcerror^2
  }
  rmse = sqrt(sse/nholdout)
  return(list(rmse=rmse, fc=fc_vec))
}

#' @description forecast rule: iid by period
iidbyperiod_fc = function(train, holdout, periodicity)
{ ntrain = length(train)
  nholdout = length(holdout)
  m = periodicity
  sse = 0
  fc_vec = numeric(nholdout)
  seasonal_means = numeric(m)
  for (j in 1:m)
  { period_indices = seq(j, ntrain, by=m)
    seasonal_means[j] = mean(train[period_indices])
  }
  for (i in 1:nholdout)
  { yt = holdout[i]
    period_index = ((ntrain+i-1) %% m) + 1
    fc = seasonal_means[period_index]
    fc_vec[i] = fc
    fcerror = yt - fc
    sse = sse + fcerror^2
  }
  return(list(rmse=sqrt(sse/nholdout), fc=fc_vec))
}

#' @description forecast rule: Winters additive seasonal exponential smoothing
#' @param train training set (ts object) with length ntrain
#' @param holding holdout set with length nholdout
#' @param alpha parameter for level equation
#' @param beta parameter for slope equation
#' @param gamma parameter for season equation
#' @param level last ell value in training set
#' @param slope last b  value in training set
#' @param season vector of s value in the last period in training set,
#'         length(season) = m = periodicity
#' @return list with holdout set rmse, holdout set forecast vector
aseason_fc = function(train, holdout, alpha, beta, gamma, level, slope, season)
{ nh = length(holdout)
  m = length(season)
  sse = 0
  fc_vec = numeric(nh)
  L = level; b = slope; s = season
  for (i in 1:nh)
  { fc = L + b + s[((i-1)%%m) + 1]
    fc_vec[i] = fc
    yt = holdout[i]
    fcerror = yt - fc
```

```r
    sse = sse + fcerror^2
    L_old = L
    L = alpha * (yt - s[((i-1)%%m) + 1]) + (1-alpha) * (L_old + b)
    b = beta * (L-L_old) + (1-beta) * b
    s[((i-1)%%m) + 1] = gamma * (yt-L_old-b) + (1-gamma) * s[((i-1)%%m) + 1]
  }
  return(list(rmse=sqrt(sse/nh), fc=fc_vec))
}

#' @description forecast rule AR(p)
#' @param arvec AR(p) regression coeffcieint vector for training set
#' @param mu mean or mu parameter from arima for fitting AR(p)
#' @return list with holdout set rmse, holdout set forecast vector
arp_fc = function(train,holdout,arvec,mu,iprint=F)
{ p = length(arvec)
  nholdout = length(holdout)
  ntrain = length(train)
  comb = c(train[(ntrain-p+1):ntrain],holdout)-mu
  fc_vec = rep(0,nholdout)
  for(i in 1:nholdout)
  { tem = comb[(i+p-1):i]
    fc_vec[i] = mu+sum(arvec*tem)
  }
  mse = mean((holdout-fc_vec)^2)
  if(iprint) print(cbind(holdout,fc_vec))
  list(rmse=sqrt(mse), fc=fc_vec)
}



# Winters additive, train is a ts object with frequency=m=periodicity
wa = HoltWinters(train,seasonal="additive")

alpha = wa$alpha;  beta = wa$beta; gamma = wa$gamma
level = wa$coef[1];  slope = wa$coef[2]; season = wa$coefficient[3:367]

cat(alpha, beta, gamma,"\n")
#> 0.8635823 0 0.6966282
cat(level, slope,"\n")
#> 6.835318 0.001230514
# Beginning of year
cat(round(season[1:10],3))
#> -9.019 -9.593 -11.058 -13.229 -12.886 -13.992 -12.655 -12.901 -12.537 -11.496
# Early July
cat(round(season[185:194],3))
#> 8.852 9.409 10.707 11.109 11.403 11.759 11.415 10.13 10.46 10.386
# End of year
cat(round(season[356:365],3))
#> -15.491 -14.366 -12.943 -11.602 -9.814 -9.96 -9.971 -11.22 -10.362 -9.487

# Get forecast vectors and rmse for 3 methods
aseason = aseason_fc(train,holdout,alpha,beta,gamma,level,slope,season)
persist_season = persistbyperiod_fc(train,holdout, periodicity=365)
```

```r
iid_season = iidbyperiod_fc(train,holdout, periodicity=365)

aseason_rmse = as.numeric(aseason$rmse)
rmse_vec = c(aseason_rmse, persist_season$rmse, iid_season$rmse)
print(round(rmse_vec,3))
#> [1] 4.166 5.995 4.409

# These will be compared with the regression method below.

out = cbind(holdout, aseason$fc, persist_season$fc, iid_season$fc)
colnames(out) = c("holdout","add_seasonal","persist_season","iid_season")
print(round(out[1:12,],2))
#>       holdout add_seasonal persist_season iid_season
#>  [1,]    -0.1        -2.18           -0.6      -4.81
#>  [2,]     3.8        -0.96            2.5      -5.43
#>  [3,]     6.2         1.69            2.0      -6.34
#>  [4,]     3.6         3.41           -0.4      -6.71
#>  [5,]     2.2         3.92            1.8      -5.87
#>  [6,]     4.1         1.33            4.0      -5.75
#>  [7,]     2.0         5.06            6.8      -4.86
#>  [8,]    -0.9         2.17            3.1      -4.59
#>  [9,]    -5.0        -0.12           -0.7      -4.78
#> [10,]    -3.9        -3.29            2.4      -4.50
#> [11,]    -3.3        -4.67            1.9      -4.65
#> [12,]    -1.1        -2.83            4.3      -3.89
```

Identities for cosine of the sum of two angles

$$\cos(\omega_1 + \omega_2) = \cos(\omega_1)\cos(\omega_2) - \sin(\omega_1)\sin(\omega_2)$$

Let ω = constant phase angle, m = periodicity.

$$\beta\cos(\omega + 2\pi t/m) = \{\beta\cos(\omega)\}\cos(2\pi t/m) + \{-\beta\sin(\omega)\}\sin(2\pi t/m) = \beta_1\cos(2\pi t/m) + \beta_2\sin(2\pi t/m)$$

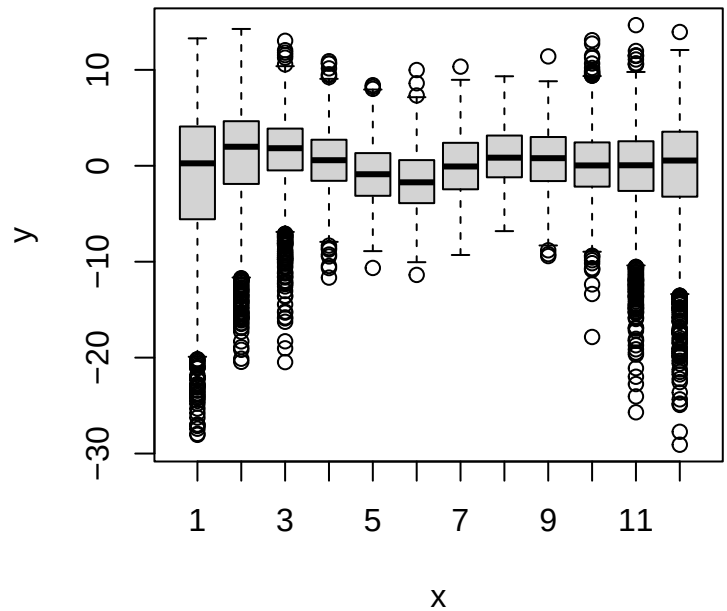
For (linear) regression), the latter form fits better than the form with the phase angle.

Step 4: Get acf, apply seasonal arma and regression on sinusoid

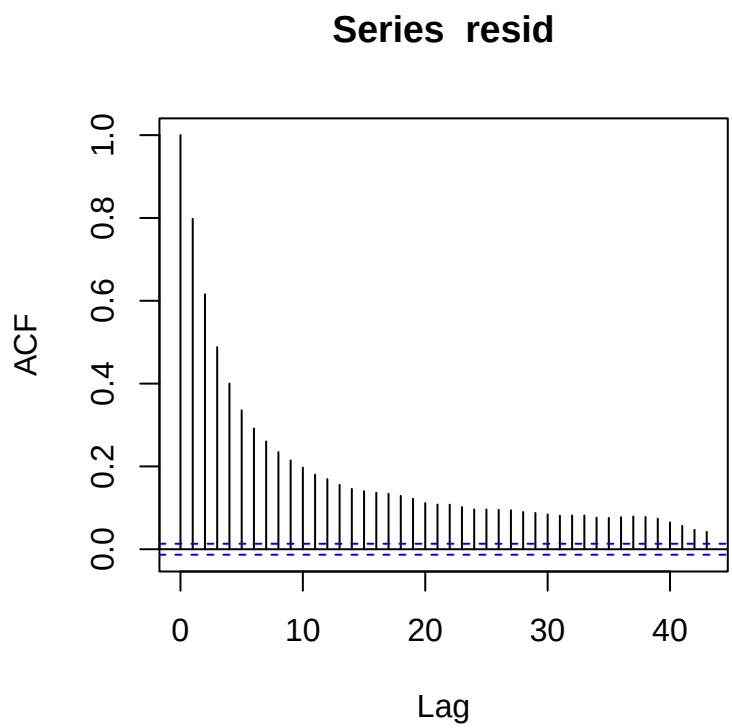
```r
acf_vec = c(acf(train,plot=F,lag.max=3*365)$acf); acf_vec = acf_vec[-1]
pacf_vec = c(pacf(train,plot=F,lag.max=3*365)$acf)

ii = c(1:10, 177:187, 360:370, 365+(361:370), 730+(360:365))
ii1 = ii[1:24] ; ii2=ii[25:48]

tem = cbind(ii1,acf_vec[ii1],pacf_vec[ii1], ii2,acf_vec[ii2],pacf_vec[ii2])
colnames(tem) = c("lag","acf","pacf", "Lag","Acf","Pacf")
print(round(tem,3))
#>       lag    acf   pacf  Lag   Acf   Pacf
#>  [1,]   1  0.958  0.958  363 0.791  0.004
#>  [2,]   2  0.920  0.026  364 0.791  0.009
#>  [3,]   3  0.893  0.117  365 0.791 -0.002
#>  [4,]   4  0.874  0.096  366 0.790 -0.002
#>  [5,]   5  0.859  0.077  367 0.789  0.004
```

```
#>  [6,]   6  0.849  0.080  368 0.789  0.012
#>  [7,]   7  0.841  0.065  369 0.789  0.009
#>  [8,]   8  0.834  0.051  370 0.789  0.014
#>  [9,]   9  0.828  0.050  726 0.774  0.022
#> [10,]  10  0.822  0.039  727 0.775 -0.006
#> [11,] 177 -0.774 -0.007  728 0.776  0.001
#> [12,] 178 -0.775 -0.013  729 0.776 -0.003
#> [13,] 179 -0.776 -0.006  730 0.775 -0.006
#> [14,] 180 -0.777 -0.006  731 0.775  0.006
#> [15,] 181 -0.777  0.001  732 0.774 -0.016
#> [16,] 182 -0.777 -0.010  733 0.773  0.002
#> [17,] 183 -0.776  0.006  734 0.772  0.003
#> [18,] 184 -0.775 -0.011  735 0.771 -0.005
#> [19,] 185 -0.775 -0.006 1090 0.774  0.024
#> [20,] 186 -0.775 -0.012 1091 0.776 -0.003
#> [21,] 187 -0.775 -0.013 1092 0.777  0.004
#> [22,] 360  0.790  0.007 1093 0.777  0.002
#> [23,] 361  0.791  0.003 1094 0.777  0.013
#> [24,] 362  0.791  0.000 1095 0.779  0.020

# arima (auto.arima) and regression on sinusoid

# Next block is commented out because it took time to run.
# library(forecast)
# temp_auto = auto.arima(train,seasonal=T,stationary=T) # takes a while
# print(temp_auto)
# Series: train
# ARIMA(5,0,0) with non-zero mean
# Coefficients:
#          ar1      ar2     ar3     ar4     ar5    mean
#       0.9116  -0.0778  0.0339  0.0253  0.0769  8.6108
# s.e.  0.0067   0.0091  0.0091  0.0091  0.0067  0.6240
#
# sigma^2 = 7.738:  log likelihood = -53478.08
# AIC=106970.1   AICc=106970.2   BIC=107026.1

train_ar5 = arima(train,order=c(5,0,0),method="ML",include.mean=T)
print(train_ar5)
#>
#> Call:
#> arima(x = train, order = c(5, 0, 0), include.mean = T, method = "ML")
#>
#> Coefficients:
#>          ar1      ar2     ar3     ar4     ar5  intercept
#>       0.9116  -0.0778  0.0339  0.0253  0.0769     8.6107
#> s.e.  0.0067   0.0091  0.0091  0.0091  0.0067     0.6241
#>
#> sigma^2 estimated as 7.736:  log likelihood = -53478.08,  aic = 106970.1

print(train_ar5$coef)
#>         ar1         ar2         ar3         ar4         ar5   intercept
#>  0.91160732 -0.07783460  0.03393027  0.02533825  0.07690337  8.61073051
```

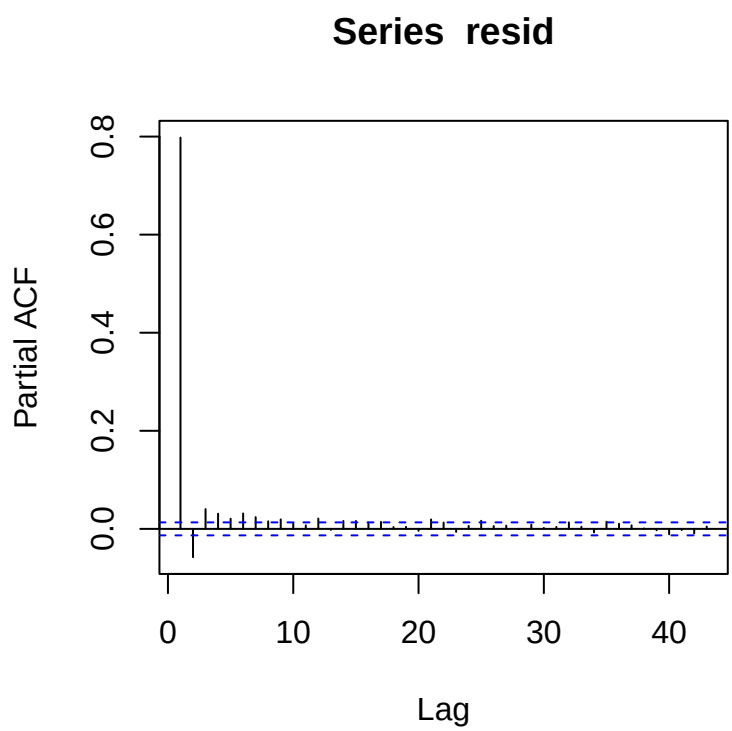
```r
arp = arp_fc(train,holdout,arvec=train_ar5$coef[1:5],mu=train_ar5$coef[6])

# rmse is printed out below to compare with other forecasting methods

# Set up for dataframes for regression with cosine, sin terms
day_train = rep(1:365,60)
day_holdout = rep(1:365,20)
day_all = rep(1:365,80)

train_df = data.frame(meantemp=train, day=day_train,
   cosine=cos(2*pi*day_train/365), sine=sin(2*pi*day_train/365))
holdout_df = data.frame(meantemp=holdout, day=day_holdout,
   cosine=cos(2*pi*day_holdout/365), sine=sin(2*pi*day_holdout/365))
all_df = data.frame(meantemp=kamloops_df$meantemp, day=day_holdout,
   cosine=cos(2*pi*day_holdout/365), sine=sin(2*pi*day_holdout/365))

reg_cossin = lm(meantemp ~ cosine+sine, data=train_df)
summary(reg_cossin)
#>
#> Call:
#> lm(formula = meantemp ~ cosine + sine, data = train_df)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -29.0799  -2.3965   0.3086   2.9397  14.6606
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   8.61082    0.03028  284.34   <2e-16 ***
#> cosine      -12.08128    0.04283 -282.10   <2e-16 ***
#> sine         -2.82254    0.04283  -65.91   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 4.481 on 21897 degrees of freedom
#> Multiple R-squared:  0.7931, Adjusted R-squared:  0.7931
#> F-statistic: 4.196e+04 on 2 and 21897 DF,  p-value: < 2.2e-16

resid = reg_cossin$residuals

# Need month of year to check on SD of residuals by month
mon_length = c(31,28,31,30,31,30,31,31,30,31,30,31)
mon = NULL
for(ii in 1:12) { mon=c(mon,rep(ii,mon_length[ii])) }

mon_train = rep(mon,60)
mon_holdout = rep(mon,20)

plot(factor(mon_train),resid)
```




```
# Looks like heteroscedastic over months.
# More variation in winter months.
# For prediction and forecasting, intervals based on homescedasticity
#   will be too long for summer months and too short for winter months.
# Can use SD(Jan)/avg(SDbymonth) etc for scaling

SDbymon = tapply(resid,mon_train,sd)
print(round(SDbymon,3))
#>     1     2     3     4     5     6     7     8     9    10    11    12
#> 7.437 5.426 3.891 3.178 3.182 3.196 3.317 2.964 3.143 3.451 4.800 6.004

pred_holdout = predict(reg_cossin, newdata=holdout_df)

reg_rmse = sqrt(mean((holdout-pred_holdout)^2))

rmse_vec = c(aseason_rmse, persist_season$rmse, iid_season$rmse, arp$rmse, reg_rmse)
print(round(rmse_vec,3))
#> [1] 4.166 5.995 4.409 2.687 4.424

# Regression on sinusoid us worse than Winters additive, slightly worse than iidbyperiod.
# AR(p) is better.

# Check residual time series for serial dependence
acf(resid)
```


Series resid
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag

```r
pacf(resid)
```


Series resid
Partial ACF
0.0
0.2
0.4
0.6
0.8
0
10
20
30
40
Lag

```r
# These plots suggest that correlated residuals so can improve by
# fitting AR(1) model to residual time series.

library(forecast)

resid_auto = auto.arima(resid,stationary=T,seasonal=F)

print(resid_auto)
#> Series: resid
#> ARIMA(3,0,1) with zero mean
#>
#> Coefficients:
#>          ar1      ar2     ar3      ma1
#>       1.7526  -0.8515  0.0861  -0.9126
#> s.e.  0.0159   0.0159  0.0070   0.0146
#>
#> sigma^2 = 7.226:  log likelihood = -52728.52
#> AIC=105467.1   AICc=105467.1   BIC=105507

acfv = c(acf(resid,plot=F,lag.max=12)$acf)
print(round(acfv,3))
#>  [1] 1.000 0.798 0.616 0.488 0.400 0.336 0.292 0.260 0.235 0.215 0.197 0.180
#> [13] 0.169

pacfv = c(pacf(resid,plot=F,lag.max=12)$acf)
print(round(pacfv,3))
#>  [1]  0.798 -0.058  0.040  0.031  0.021  0.031  0.024  0.015  0.019  0.012
#> [11]  0.007  0.021

# Use simpler AR(1) model for residuals

resid_ar1 = arima(resid, order=c(1,0,0), include.mean=F)
print(resid_ar1)
#>
#> Call:
#> arima(x = resid, order = c(1, 0, 0), include.mean = F)
#>
#> Coefficients:
#>          ar1
#>       0.7979
#> s.e.  0.0041
#>
#> sigma^2 estimated as 7.296:  log likelihood = -52837.17,  aic = 105678.4

phi = resid_ar1$coef
bvec = reg_cossin$coef

pred_holdout = predict(reg_cossin,newdata=holdout_df)
resid_holdout = holdout-pred_holdout
summary(resid_holdout)
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> -23.410  -1.334   1.364   1.071   3.886  15.585
```

```r
ntrain = length(train)
nholdout = length(holdout)
resid_trainlast = resid[ntrain]

resid_prev = c(resid_trainlast,resid_holdout[-nholdout])
cor(resid_holdout,resid_prev)
#> [1] 0.7949988

reg_ar1_fc = pred_holdout + phi*resid_prev

reg_ar1_rmse = sqrt(mean((holdout-reg_ar1_fc)^2))
rmse_vec = c(aseason_rmse, persist_season$rmse, iid_season$rmse, arp$rmse,
   reg_rmse, reg_ar1_rmse)
print(round(rmse_vec,3))
#> [1] 4.166 5.995 4.409 2.687 4.424 2.613

# Regression combined with AR(1) for residual series is better.
```

Step 5: ARMAX with sinusoid as predictor variables, and AR(1) for residual process

```r
# Last step : ARMAX with xreg=(cos,sin) and specify order as (1,0,0)
# or ARX with AR(1) on residuals

train_arx = arima(train_df$meantemp,order=c(1,0,0), xreg=train_df[,3:4], method="CSS")
print(train_arx)
#>
#> Call:
#> arima(x = train_df$meantemp, order = c(1, 0, 0), xreg = train_df[, 3:4], method = "CSS")
#>
#> Coefficients:
#>          ar1  intercept    cosine     sine
#>       0.7979     8.6108  -12.0813  -2.8225
#> s.e.  0.0041     0.0903    0.1274   0.1274
#>
#> sigma^2 estimated as 7.297:  part log likelihood = -52837.16

names(train_arx)
#>  [1] "coef"      "sigma2"    "var.coef"  "mask"      "loglik"    "aic"
#>  [7] "arma"      "residuals" "call"      "series"    "code"      "n.cond"
#> [13] "nobs"      "model"
print(train_arx$coef)
#>         ar1   intercept      cosine        sine
#>   0.7979028   8.6108240 -12.0812835  -2.8225427

# Compare with previous two-step estimates
print(reg_cossin$coef)
#> (Intercept)      cosine        sine
#>    8.610824  -12.081283   -2.822543
print(resid_ar1$coef)
#>     ar1
#> 0.79789
```

```r
#' @description
#' Moving 1-step ahead forecasts for holdout set, seasonal ARMA
#' @param tsdata vector with training and holdout data
#' @param ntrain length of training set
#' @param order ARIMA order using for training set
#' @param method as used for arima applied to training set
#' @param traincoef the $coef component for arima applied to training set
#' @param include.mean flag as used for arima applied to training set
#' @param xreg matrix with predictors for arima()
#' @return holdout set rmse and 1-step forecasts
armax_fc = function(tsdata,ntrain,order,method,traincoef,include.mean,xreg,
   iprint=F)
{ # obj = arima(tsdata,order=order,init=traincoef,fixed=traincoef,
  # next line for debugging, after debugging, replace with above line
  obj <<- arima(tsdata,order=order,init=traincoef,fixed=traincoef,
     method=method, include.mean=include.mean, xreg=xreg,
     optim.control=list(maxit=0))
  # yt = y_{t|t-1} + innov ; fc = yt-innov
  fc = tsdata - obj$residuals
  ntotal = length(tsdata)
  holdout_fc = fc[(ntrain+1):ntotal]
  holdout = tsdata[(ntrain+1):ntotal]
  if(iprint) print(cbind(holdout,holdout_fc))
  rmse = sqrt(mean((holdout-holdout_fc)^2))
  list(rmse=rmse, fc=holdout_fc)
}

ntrain = nrow(train_df)
ntotal = nrow(all_df)

# Previous use of arima()
# train_arx = arima(train_df$meantemp,order=c(1,0,0),xreg=train_df[,3:4],method="CSS")

xreg = armax_fc(all_df$meantemp, ntrain=ntrain, order=c(1,0,0), method="CSS",
  traincoef=train_arx$coef, include.mean=T, xreg=all_df[,3:4])
rmse_vec = c(aseason_rmse, persist_season$rmse, iid_season$rmse, reg_rmse,
   reg_ar1_rmse, xreg$rmse)
print(round(rmse_vec,3))
#> [1] 4.166 5.995 4.409 4.424 2.613 2.613

# Matches close to 2-step estimation approach
```

```r
# Debugging/validation
head(obj$residuals)
#> Time Series:
#> Start = 1
#> End = 6
#> Frequency = 1
#> [1]  0.0000000  6.6881569  0.6069130  0.7294092  0.4117927 -0.6469863

obj$residuals[(ntrain-5):ntrain]
#> [1] -0.91424441 -2.78902377 -0.44746462 -2.98013239  3.20143572 -0.04346846

length(train_arx$residuals)
#> [1] 21900

print(ntrain)
#> [1] 21900

head(train_arx$residuals)
#> Time Series:
#> Start = c(1, 1)
#> End = c(1, 6)
#> Frequency = 365
#> [1]  0.0000000  6.6881569  0.6069130  0.7294092  0.4117927 -0.6469863

tail(train_arx$residuals)
#> Time Series:
#> Start = c(60, 360)
#> End = c(60, 365)
#> Frequency = 365
#> [1] -0.91424441 -2.78902377 -0.44746462 -2.98013239  3.20143572 -0.04346846

# Matches to training set

save(file="03kamloops.RData", train, holdout, train_df, holdout_df, all_df,
  train_ar5, reg_cossin, resid_ar1, train_arx)
```

Final checks for ARX two-step and one-step, n.ahead forecasts

```r
compare = cbind(reg_ar1_fc,xreg$fc)
head(compare)
#>   reg_ar1_fc
#> 1 -2.9025129 -2.9025032
#> 2 -0.8338630 -0.8338193
#> 3  2.2727832  2.2728773
#> 4  4.1833350  4.1834603
#> 5  2.1051787  2.1052713
#> 6  0.9852335  0.9853086


# Check how predct.Arima() works with xreg
```

```r
nahead = 5
angle = 2*pi*(1:nahead)/365
xregnew1 = cbind(cos(angle), sin(angle))
pred1 = predict(train_arx, n.ahead=nahead, se.fit=T, newxreg=xregnew1)

print(pred1)
#> $pred
#> Time Series:
#> Start = c(61, 1)
#> End = c(61, 5)
#> Frequency = 365
#> [1] -2.902503 -3.069944 -3.208671 -3.323744 -3.419204
#>
#> $se
#> Time Series:
#> Start = c(61, 1)
#> End = c(61, 5)
#> Frequency = 365
#> [1] 2.701256 3.455760 3.860029 4.096674 4.240460

# Pretend beginning of February
angle = 2*pi*(32:(31+nahead))/365
xregnew2 = cbind(cos(angle), sin(angle))
pred2 = predict(train_arx, n.ahead=nahead, se.fit=T, newxreg=xregnew2)

print(pred2)
#> $pred
#> Time Series:
#> Start = c(61, 1)
#> End = c(61, 5)
#> Frequency = 365
#> [1] -2.545977 -2.601022 -2.627491 -2.630479 -2.614059
#>
#> $se
#> Time Series:
#> Start = c(61, 1)
#> End = c(61, 5)
#> Frequency = 365
#> [1] 2.701256 3.455760 3.860029 4.096674 4.240460
```

Note: Heteroscedasticity does not prevent holdout set forecast RMSE. However prediction intervals would be be overestimated in length for summer months and underestimated for winter months.